



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



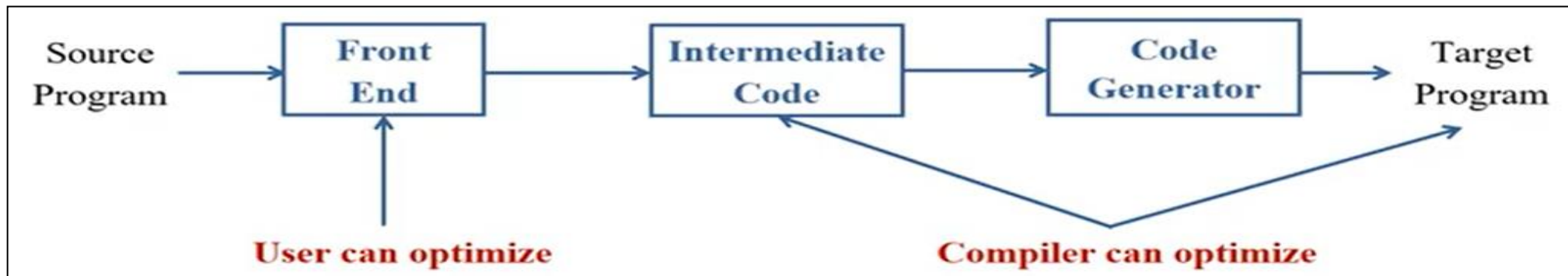
5

CODE OPTIMIZATION

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

CODE OPTIMIZATION

- The code optimization in the synthesis phase is a program transformation technique, which tries to improve the intermediate code by making it consume fewer resources (i.e. CPU, Memory) so that faster-running machine code will result.
- Compiler optimizing process should meet the following objectives :
 - ***The optimization must be correct, it must not, in any way, change the meaning of the program.***
 - ***Optimization should increase the speed and performance of the program.***
 - ***The compilation time must be kept reasonable.***
 - ***The optimization process should not delay the overall compiling process.***



- Optimization can be categorized into two types:

Machine Independent Optimization

Machine Dependent Optimization

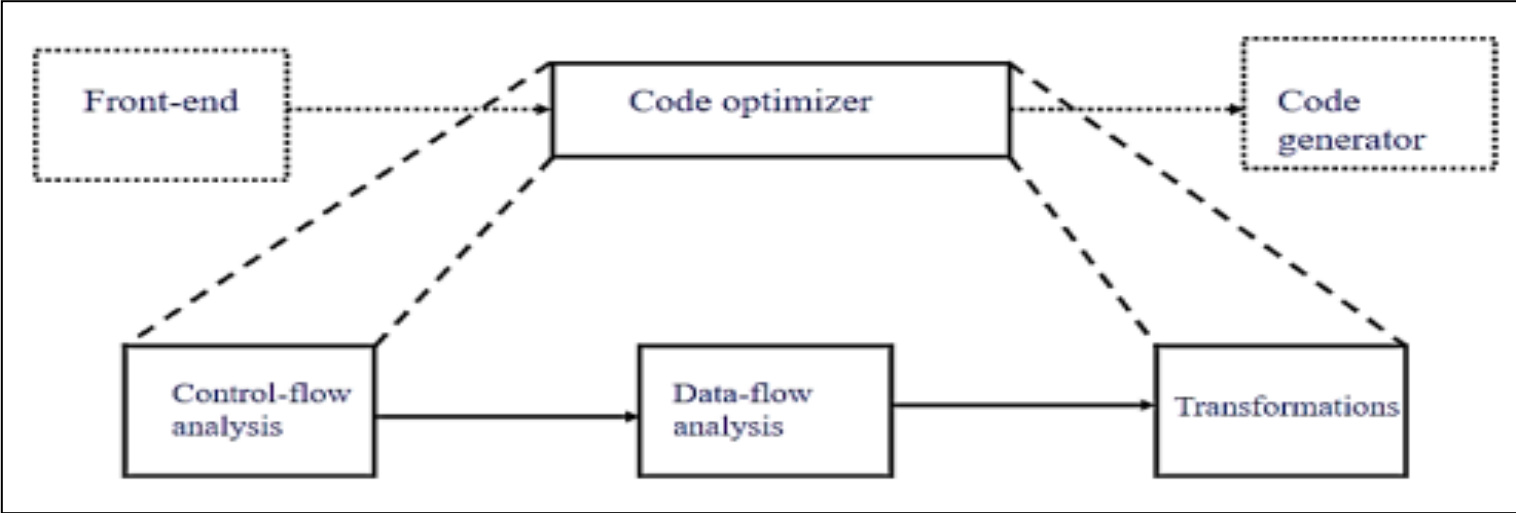
Types	Description
Machine Independent Optimization	• This code optimization phase attempts to improve the intermediate code to <i>get a better target</i> code as the output. • The part of the intermediate code which is transformed here <i>does not involve</i> any CPU registers or absolute memory locations
Machine Dependent Optimization	• Machine-dependent optimization is done <i>after the target code</i> has been generated and when the code is transformed according to the target machine architecture. • It <i>involves</i> CPU registers and may have absolute memory references rather than relative references.

Organization Of Code Optimizer

- Optimization can be done in 2 phases

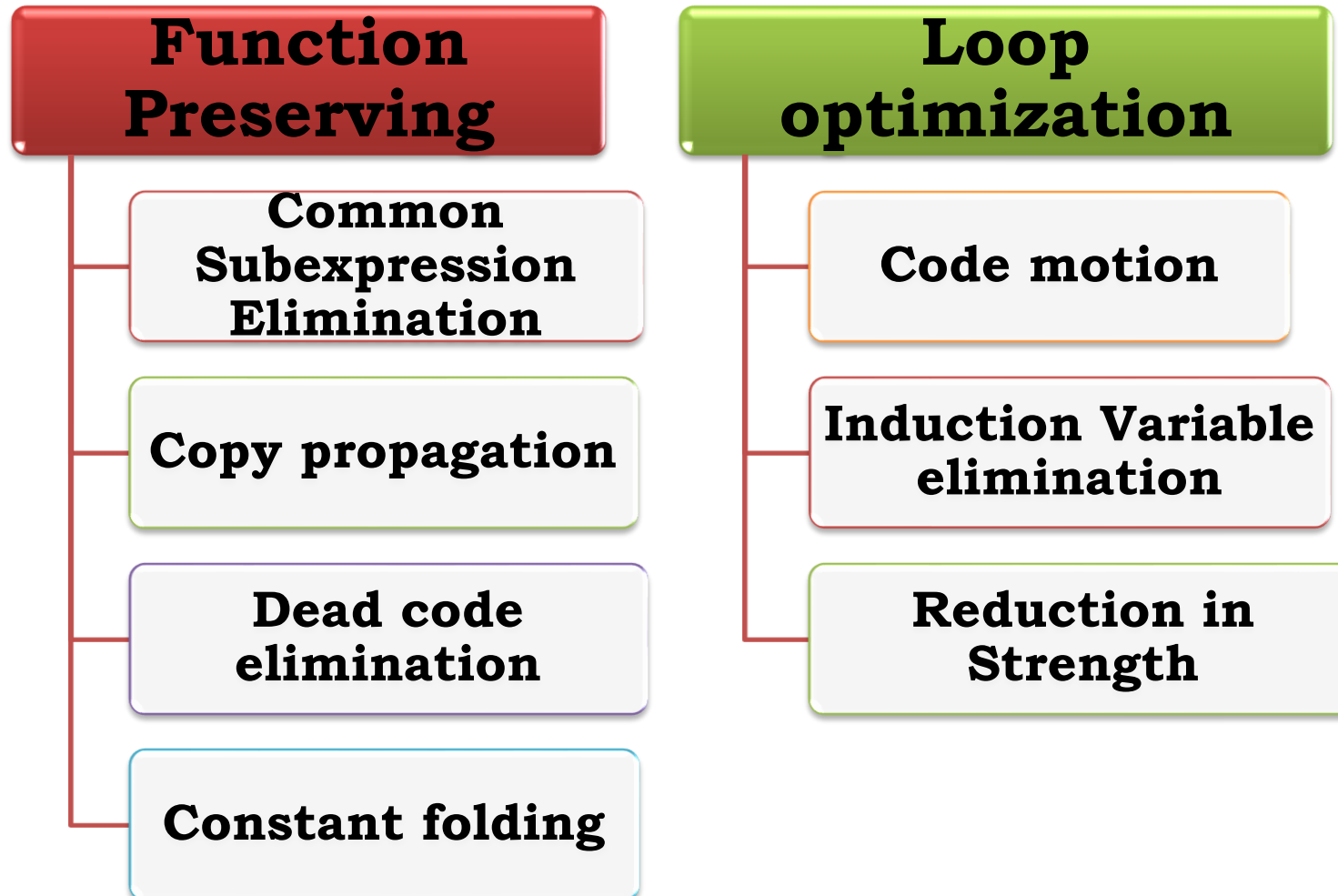
Phases	Description
Local optimization	• Transformations are applied over a <i>small segment</i> of the program called basic block, in which statements are executed in <i>sequential order</i>. • Speed-up factor for local optimization is 1.4.
Global optimization	• Transformations are applied over <i>a large segment</i> of the program like loop, procedures, functions etc. • Local optimization to be done before applying global optimization. • Speed-up factor is 2.7.

- **Control-Flow Analysis:** *Identifies loops* in the flow graph of a program since such loops are usually good candidates for improvement.
- **Data-Flow Analysis:** Collects information about the way *variables* are used in a program.



Principal Sources of Optimization

- Machine independence includes two types
 - Function Preserving
 - Loop optimization



(i) Function-Preserving optimization

- It can improve the code optimization *without changing the function* it computes.
- a. Common Sub Expression Elimination (CSE)**
 - The expression that has been already computed before and appears again in the code for computation is called as **Common Sub-Expression**.
 - Elimination of that sub-expression is known as **Common sub-expression elimination**.
 - The redundant expressions are eliminated to avoid their re-computation.
 - The two types of elimination methods in common sub-expression elimination are:
 - **1. Local Common Sub-expression elimination**
 - It is used within a single basic block, where a basic block is a simple code sequence that has no branches.
 - **2. Global Common Sub-expression elimination**
 - It is used for an entire procedure of common sub-expression elimination.

Before elimination

```
a = 10;  
b = a + 1 * 2;  
c = a + 1 * 2;  
// 'c' has common expression as 'b'  
d = c + a;
```



After elimination

```
a = 10;  
b = a + 1 * 2;  
d = b + a;
```

Before elimination

```
x = 11;  
y = 11 * 24;  
z = x * 24;  
// 'z' has common expression as 'y' as 'x' can be  
evaluated directly done in 'y'.
```



After elimination

```
y = 11 * 24;
```

Before elimination

```
t1: = 4*i  
t2: = a [t1]  
t3: = 4*j  
t4: = 4 * i  
t5: = n  
t6: = b [t4] + t5
```



After elimination

```
t1: = 4*i  
t2: = a [t1]  
t3: = 4*j  
t5: = n  
t6: = b [t1] + t5
```

Before

```
a = b + c  
b = a - d  
c = b + c  
d = a - d
```



After

```
a = b + c  
b = a - d  
c = b + c  
d = b
```

b. Copy propagation

- Copy propagation means *use of one variable instead of another*.
- Copy statements introduced during common subexpression elimination.
- Assignments of the form $\mathbf{f} := \mathbf{g}$ called copy statements, or copies for short.
- The idea behind the copy-propagation transformation is to use $\mathbf{g}$ for $\mathbf{f}$, whenever possible after the copy statement $\mathbf{f} := \mathbf{g}$.

Before Copy propagation

```
T1 = X
T2=T3+T2
T3=X
```

After Copy propagation

```
T2 = T3 + T2
T3 = T1
```

Before Copy propagation

```
Var1 = a+b    // assigning value to the variable
Var2 = Var1
A = Var2+B
```

After Copy propagation

```
Var1 = a+b
A = Var1+B
```


c. Dead code elimination

- Dead code is a program snippet that is *never executed* or *never reached* in a program.
- It is a code that can be efficiently removed from the program without affecting any other part of the program.
- In case, a value is obtained and never used in the future, it is also regarded as dead code.

Before Optimization	After Optimization
<pre>i=0; if(i==1) { a=b+5; }</pre>	<pre>i=0; // 'if' statement is dead code because this condition will never get satisfied.</pre>
<pre>int add(int a,int b) { int x,y,z; return (a+b); printf("hello compiler"); printf("how are you"); }</pre>	<pre>int add(int a,int b) { return (a+b); }</pre>

d. Constant folding

- Constant folding in compiler design is a compiler optimization technique that **eliminates expressions of the code** whose **value can be computed before executing the code**.
- Multiple constants are folded together and evaluated at the compile time.
- Constant folding can be applied for the following data types:
 - **Boolean values**
 - **Integers**, except for division by zero exception.
 - **Floating point values**, *with caution while rounding*.

```
//Code segment  
int x = 5+7+c;  
  
//Folding applied  
int x = 12+c;
```

```
//Code segment  
int z = 300 * 78 * 6  
//Folding applied  
int z = 140400
```

```
//Code segment  
bool flag = a & false;  
//Folding applied  
bool flag = false;
```

```
//Code segment  
int f (void)  
{  
  return 3 + 5;  
}  
  
//Folding applied  
int f (void)  
{  
  return 8;  
}
```

(ii) Loop optimization

- The optimization performed on *inner loops* is called loop optimization.
 - Inner loop is a place where program spends large amount of time.
 - If number of instructions is less in inner loop the running time of the program decreases
- Loop Optimization is the process of *increasing execution speed* and *reducing the overheads* associated with loops.
- Loop Optimization is a *machine independent optimization*.
- It plays an important role in improving *cache performance* and making effective use of *parallel processing capabilities*.

a. Code Motion (Frequency Reduction or Loop Invariant)

- In frequency reduction, *the amount of code in the loop is decreased.*

- A statement or expression, which can be moved outside the loop body without affecting the semantics of the program, is moved outside the loop.

Before Optimization	After Optimization
<pre>for (int i = 0 ; i < n ; i++) { x = y + z; a[i] = b * i }</pre>	<pre>x = y + z; for (int i = 0 ; i < n ; i++) { a[i] = b * i }</pre>
<pre>while(i<=limit-2)</pre>	<pre>t :=limit-2; while(i<=t)</pre>
<pre>while(i<100) { a = Sin(x)/Cos(x) + i; i++; }</pre>	<pre>t = Sin(x)/Cos(x); while(i<100) { a = t + i; i++; }</pre>

- A fragment of code that resides in the loop and computes the same value of each iteration is called loop invariant code

Before code motion

```
p=100
for(i=0;i<p;i++)
{
    a=b+40;           //loop invariant code
    if(p/a==0)
        printf("%d",p);
}
```

After code motion

```
p=100
a=b+40;
for(i=0;i<p;i++)
{
    if(p/a==0)
        printf("%d",p);
}
```

- Before optimizing, the loop invariant code was evaluated for every iteration of the loop.
- Once code motion is applied, the frequency of evaluating loop invariant code also decreases.

b. Induction Variable elimination

- Any two variables are said to be induction variables ,*if there is a change in any one of the variable, then there is a corresponding change in the other variable.*
- The aim of the induction variable elimination method is to replace the induction variable from the loop.
- With each iteration, its value either gets incremented or decremented by some constant value.
- This technique *improves the execution time of a program.*

```
int i, maz = 10, r;  
    r = max-1;  
        for(i=10;i<=r;i++)  
        {  
            printf("%d", i);  
        }
```

Before elimination

```
for(i=1;i<=11;++i)  
{  
    j=i*5;  
}
```

After elimination

```
j=0;  
for(i=1;i<=11;++i)  
{  
    j=j+5;  
}
```

variable i is called induction variable as values of i get incremented by 1, i.e., 0,1,2,3,4,5,6,7,8,9,10

c. Reduction in Strength

- Strength reduction or reduction in strength is a technique in which we replace expensive operations with cheaper operations.
- Strength reduction means replacing the high strength operator with a low strength.
- Strength of * is higher than +.
- Usually, compiler takes more time for higher strength operators and execution speed is less.

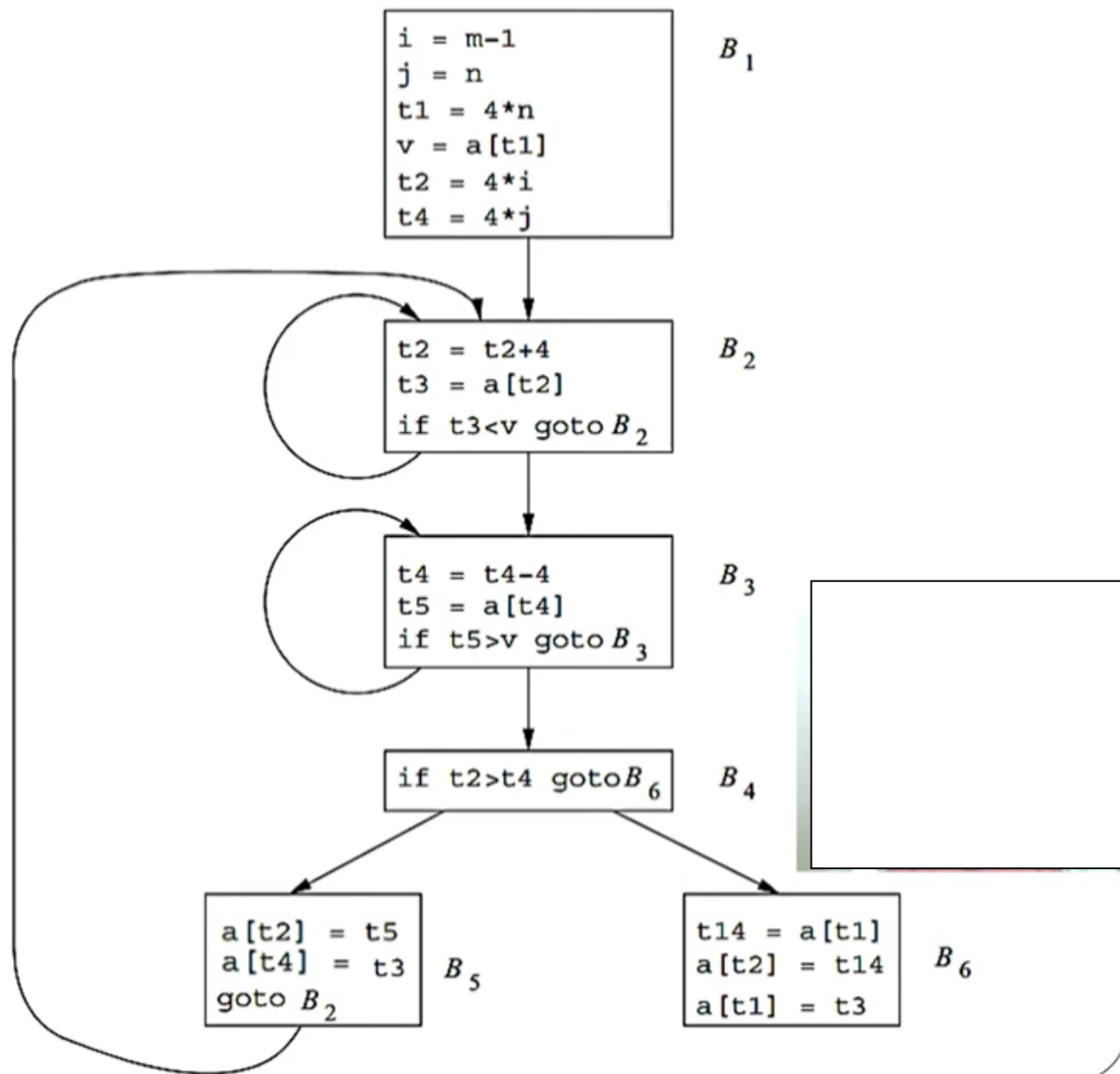
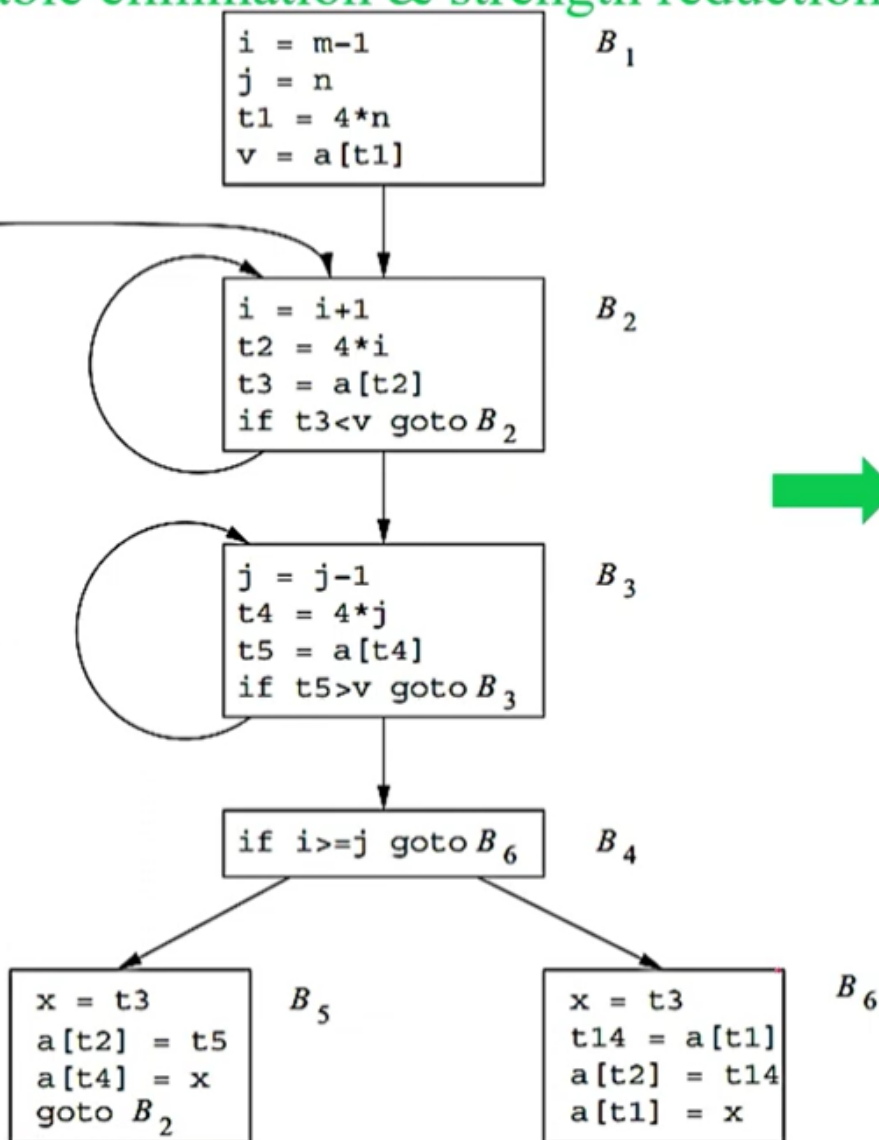
Before optimization

```
for (i=1;i<=10;i++)  
{  
    sum = i * 7;  
    printf("%d", sum);  
}
```

After optimization

```
temp = 7;  
for(i=1;i<=10;i++)  
{  
    temp = temp + 7;  
    sum = temp;  
    printf("%d", sum)  
}
```

- Flow graph after Copy Propagation , Dead-Code Elimination , Code Motion, induction-variable elimination & strength reduction



```
int sum = 0;
int x = 2;
for (int i = 0; i < 100; i++)
{
    sum += i * x * 5;
}
```

Code Motion

```
int sum = 0;
int x = 2;
int temp = x * 5;
for (int i = 0; i < 100; i++)
{
    sum += i * temp;
}
```

Induction Variable Elimination

```
int sum = 0;
int x = 2;
int temp = x * 5;
int acc = 0;
for (int i = 0; i < 100; i++)
{
    sum += acc;
    acc += temp;
}
```

Reduction in Strength

```
int sum = 0;
int x = 2;
int temp = x * 5;
for (int i = 0; i < 100; i++)
{
    sum += (i << 1) + (i << 3);
}
```

```
#include <stdio.h>

void compute(int n)
{
    int a[n], b[n], sum = 0;

    for (int i = 0; i < n; i++)
    {
        a[i] = i * 2;
    }
    for (int i = 0; i < n; i++)
    {
        b[i] = a[i] + 1;
        sum += b[i];
    }

    printf("Sum: %d\n", sum);
}
```

Code Motion

```
#include <stdio.h>

void compute(int n)
{
    int a[n], b[n], sum = 0;

    for (int i = 0; i < n; i++)
    {
        a[i] = i * 2;
    }
    int inc = 1;
    for (int i = 0; i < n; i++)
    {
        b[i] = a[i] + inc;
        sum += b[i];
    }

    printf("Sum: %d\n", sum);
}
```

Induction Variable Elimination

```
#include <stdio.h>

void compute(int n) {
    int a[n], b[n], sum = 0;

    for (int i = 0; i < n; i++)
    {
        a[i] = i * 2;
    }

    for (int i = 0; i < n; sum += b[i++])
    {
        b[i] = a[i] + 1;
    }

    printf("Sum: %d\n", sum);
}
```

Peephole Optimization

- **Peephole** is a short sequence of target instruction.
 - The peephole can be considered as a moving window on the target program.
 - The instructions in the peephole need not be contiguous.
- Peephole optimization is a **local optimization technique** that compilers use to optimize the generated code.
- It is called **local optimization** because it works by evaluating a small section of the generated code, generally a few instructions, and optimizing them based on some predefined rules.
- Peephole optimization can also be applied directly after intermediate code generation to improve the intermediate representation.
- The peephole is **machine-dependent optimization**.

- The **objectives** of peephole optimization are as follows:
 - *Improve performance*
 - *Reduce memory footprint*
 - *Reduce code size.*
- In general peephole optimization involves the following six transformations:
 - *Redundant instruction elimination*
 - *Unreachable code elimination*
 - *Flow of control optimization*
 - *Algebraic simplification*
 - *Reduction in strength*
 - *Use of machine idioms*

- **Working of Peephole Optimization in Compiler design**

Identify the peephole

- In the first step, the compiler finds the **small sections of the generated code** that needs optimization.

Apply the optimization rule

- After identification, in the second step, the compiler applies a **predefined set of optimization rules** to the instructions in the peephole.

Evaluate the result

- After applying optimization rules, the compiler evaluates the optimized code to check whether the changes make the code better than the original in terms of speed, size, or memory usage

Repeat

- The process is **repeated by finding new peepholes** and applying the optimization rules until no more opportunities to optimize exists.

a. Redundant load and store elimination

- Redundant load and store elimination is a peephole optimization approach that seeks to **reduce redundant memory accesses** in a program.
- This optimization works by finding code that performs the same memory access many times and removes the redundant accesses.
- At source code level, the following can be done by the user:

```
int add_ten(int x)
{
    int y, z;
    y = 10;
    z = x + y;
    return z;
}
```

```
int add_ten(int x)
{
    int y;
    y = 10;
    y = x + y;
    return y;
}
```

```
int add_ten(int x)
{
    int y = 10;
    return x + y;
}
```

```
int add_ten(int x)
{
    return x + 10;
}
```

- At compilation level, the compiler searches for instructions redundant in nature. Multiple loading and storing of instructions may carry the same meaning even if some of them are removed.

MOV x, R0
MOV R0, R1

Delete first instruction and re-write the sentence as

MOV x, R1

Example

$a = b + c$
 $d = a + e$

It is implemented on the register(R0)

Initial code

MOV b, R0;	Instruction to copy b to the register
ADD c, R0;	Instruction to Add c to the register, the register is now b+c
MOV R0, a;	Instruction to Copy the register(b+c) to a
MOV a, R0;	Instruction to Copy a to the register
ADD e, R0;	Instruction to Add e to the register, the register is now a(b+c)+e
MOV R0, d;	Instruction to Copy the register to d

This can be optimized by removing load and store operation, like in third instruction value in register R0 is copied to a, and it again loaded to R0 in the next step for further operation

Optimized code

MOV b, R0;	Instruction to Copy b to the register
ADD c, R0;	Instruction to Add c to the register, which is now b+c (a)
MOV R0, a;	Instruction to Copy the register to a
ADD e, R0;	Instruction to Add e to the register, which is now b+c+e [(a)+e]
MOV R0, d;	Instruction to Copy the register to d

b. Eliminating Unreachable Code

- Unreachable code is a part of the program code that is never accessed because of programming constructs.
- The code following an unconditional jump is unreachable and can be eliminated.
- The same operation can be repeated to eliminate a sequence of instructions

```
void add_ten(int x)  
{  
    return x + 10;  
    printf("value of x is %d", x);  
}
```

- The printf statement will never be executed as the program control returns back before it can execute, hence printf can be removed.

- Consider the following C code and its intermediate code

```
#define debug 0  
  
    . . . .  
    if (debug)  
    {  
        print debugging information  
    }
```

Intermediate representation

```
if debug == 1 goto L1  
    goto L2  
L1: print debugging information  
L2:
```

The intermediate code can be transformed as shown by eliminating jumps over jumps

```
if debug != 1 goto L2  
    print debugging information  
    L2:
```

As debug is set 0 at the beginning by constant propagation, the code is

```
if 0 != 1 goto L2  
print debugging information  
    L2:
```

As the condition is always true, it can be replaced by goto L2, and statements that print debugging information are unreachable and can be eliminated.

c. Flow-of-Control Optimizations

- There are instances in a code where the program control jumps back and forth without performing any significant task.
- The intermediate code generated, sometimes contains
 - *jumps to jumps*
 - *jumps to conditional jumps*
 - *conditional jumps to jumps*

Jumps to Jump

- Consider the following jump sequence

```
    goto L1
    . . .
L1: goto L2
    . . .
L2: a=b
```

The jump sequence can be replaced as

```
    goto L2
    . . .
L1: goto L2
    . . .
L2: a=b
```

- If there are no jumps to L1 and the statement L1: goto L2 is preceded by an unconditional jump, then the statement L1: goto L2 can be eliminated

conditional jumps to jumps

- Consider the following sequence

```
    if a < b goto L1  
L1: goto L2
```

The jump sequence can be replaced as

```
    if a < b goto L2  
L1: goto L2
```

jumps to conditional jumps

- Consider the following jump sequence

```
    goto L1  
L1: if a < b goto L2  
L3:
```

jump sequence can be replaced by the following sequence, provided there is only one jump to L1, and L2 is preceded by an unconditional jump

```
    if a < b goto L2  
    goto L3  
    ....  
L3:
```

d. Algebraic Simplification and Reduction in Strength

- The **algebraic expressions** that are useless or written inefficiently are transformed.

a=a + 0

a=a * 1

a=a / 1

a=a - 0

- All these expressions are causing calculation overhead.
- These can be removed for optimization

- In **strength reduction optimization**, operators that consume higher execution time are replaced by the operators consuming less execution time.

x ↑ 2 = x * x

2 * x = x + x

x * 2 = x << 1

x * 4 = x << 2

x * 8 = x << 3

x / 2 = x >> 1

Initial code:

y = x * 2;

Optimized code:

y = x + x;

or

y = x << 1;

e. Use of Machine Idioms

- The target machine may have some *hardware instructions* to implement certain operations efficiently.
- Using such instructions in the target code can *reduce execution time* significantly
- Example:
 - Some machine have **auto-increment** or **auto-decrement** addressing modes that are used to perform add or subtract operations.

$i = i + 1$	$\rightarrow$	INC i
$i = i - 1$	$\rightarrow$	DEC i